9 Emulation and debugging techniques

Debugging techniques

The fundamental aim of a debugging methodology is to restrict the introduction of untested software or hardware to a single item. This is good practice and of benefit to the design and implementation of any system, even those that use emulation early on in the design cycle.

It is to prevent this integration of two unknowns that simulation programs to simulate and test software, hardware or both can play a critical part in the development process.

High level language simulation

If software is written in a high level language, it is possible to test large parts of it without the need for the hardware at all. Software that does not need or use I/O or other system dependent facilities can be run and tested on other machines, such as a PC or a engineering workstation. The advantage of this is that it allows a parallel development of the hardware and software and added confidence, when the two parts are integrated, that it will work.

Using this technique, it is possible to simulate I/O using the keyboard as input or another task passing input data to the rest of the modules. Another technique is to use a data table which contains data sequences that are used to test the software.

This method is not without its restrictions. The most common mistake with this method is the use of non-standard libraries which are not supported by the target system compiler or environment. If these libraries are used as part of the code that will be transferred, as opposed to providing a user interface or debugging facility, then the modifications needed to port the code will devalue the benefit of the simulation.

The ideal is when the simulation system is using the same library interface as the target. This can be achieved by using the target system or operating system as the simulation system or using the same set of system calls. Many operating systems support or provide a UNIX compatible library which allows UNIX software to be ported using a simple recompilation. As a result, UNIX systems are often employed in this simulation role. This is an advantage which the POSIX compliant operating system Lynx offers.

This simulation allows logical testing of the software but rarely offers quantitative information unless the simulation

environment is very close to that of the target, in terms of hardware and software environments.

Low level simulation

Using another system to simulate parts of the code is all well and good, but what about low level code such as initialisation routines? There are simulation tools available for these routines as well. CPU simulators can simulate a processor, memory system and, in some cases, some peripherals and allow low level assembler code and small HLL programs to be tested without the need for the actual hardware. These tools tend to fall into two categories: the first simulate the programming model and memory system and offer simple debugging tools similar to those found with an onboard debugger. These are inevitably slow, when compared to the real thing, and do not provide timing information or permit different memory configurations to be tested. However, they are very cheap and easy to use and can provide a low cost test bed for individuals within a large software team. There are even shareware simulators for the most common processors such as the one from the University of North Carolina which simulates an MC68000 processor.

```
<D0> =00000000 <D4> =00000000 <A0> =00000000 <A4> =00000000
<D1> =00000000 <D5> =0000abcd <A1> =00000000 <A5> =00000000
<D2> =00000000 <D6> =00000000 <A2> =00000000 <A6> =00000000
<D3> =00000000 <D7> =00000000 <A3> =00000000 <A7> =00000000
trace: on    sstep: on     cycles:    416    <A7'>= 00000f00
         cn tr st rc       T S  INT   XNZVC  <PC> = 00000090
  port1  00 00 82 00  SR = 1010101111011111
-------------------------------------------------
executing a ANDI          instruction at location   58
executing a ANDI          instruction at location   5e
executing a ANDI          instruction at location   62
executing a ANDI_TO_CCR   instruction at location   68
executing a ANDI_TO_SR    instruction at location   6c
executing a OR            instruction at location   70
executing a OR            instruction at location   72
executing a OR            instruction at location   76
executing a ORI           instruction at location   78
executing a ORI           instruction at location   7e
executing a ORI           instruction at location   82
executing a ORI_TO_CCR    instruction at location   88
executing a ORI_TO_SR     instruction at location   8c
TRACE exception occurred at location   8c.
Execution halted
```

Example display from the University of North Carolina 68k simulator

The second category extends the simulation to provide timing information based on the number of clock cycles. Some simulators can even provide information on cache performance, memory usage and so on, which is useful data for making hardware decisions. Different performance memory systems can be exercised using the simulator to provide performance data. This type of information is virtually impossible to obtain

without using such tools. These more powerful simulators often require very powerful hosts with large amounts of memory. SDS provide a suite of such tools that can simulate a processor and memory and with some of the integrated processors that are available, even emulate onboard peripherals such as LCD controllers and parallel ports.

Simulation tools are becoming more and more important in providing early experience of and data about a system before the hardware is available. They can be a little impractical due to their performance limitations — one second of processing with a 25 MHz RISC processor taking 2 hours of simulation time was not uncommon a few years ago — but as workstation performance improves, the simulation speed increases. With instruction level simulators it is possible with a top of the range workstation to get simulation speeds of 1 to 2 MHz.

Onboard debugger

The onboard debugger provides a very low level method of debugging software. Usually supplied as a set of EPROMs which are plugged into the board or as a set of software routines that are combined with the applications code, they use a serial connection to communicate with a PC or workstation. They provide several functions: the first is to provide initialisation code for the processor and/or the board which will normally initialise the hardware and allow it to come up into a known state. The second is to supply basic debugging facilities and, in some cases, allow simple access to the board's peripherals. Often included in these facilities is the ability to download code using a serial port or from a floppy disk.

```
>TR

PC=000404 SR=2000 SS=00A00000 US=00000000          X=0
A0=00000000 A1=000004AA A2=00000000 A3=00000000 N=0
A4=00000000 A5=00000000 A6=00000000 A7=00A00000 Z=0
D0=00000001 D1=00000013 D2=00000000 D3=00000000 V=0
D4=00000000 D5=00000000 D6=00000000 D7=00000000 C=0
---------->LEA       $000004AA,A1

>TR

PC=00040A SR=2000 SS=00A00000 US=00000000          X=0
A0=00000000 A1=000004AA A2=00000000 A3=00000000 N=0
A4=00000000 A5=00000000 A6=00000000 A7=00A00000 Z=0
D0=00000001 D1=00000013 D2=00000000 D3=00000000 V=0
D4=00000000 D5=00000000 D6=00000000 D7=00000000 C=0
---------->MOVEQ     #19,D1

>
```

Example display from an onboard M68000 debugger

When the board is powered up, the processor fetches its reset vector from the table stored in EPROM and then starts to

initialise the board. The vector table is normally transferred from EPROM into a RAM area to allow it to be modified, if needed. This can be done through hardware, where the EPROM memory address is temporarily altered to be at the correct location for power-on, but is moved elsewhere after the vector table has been copied. Typically, a counter is used to determine a preset number of memory accesses, after which it is assumed that the table has been transferred by the debugger and the EPROM address can safely be changed.

The second method, which relies on processor support, allows the vector table to be moved elsewhere in the memory map. With the later M68000 processors, this can also be done by changing the vector base register which is part of the supervisor programming model.

The debugger usually operates at a very low level and allows basic memory and processor register display and change, setting RAM-based breakpoints and so on. This is normally performed using hexadecimal notation, although some debuggers can provide a simple disassembler function. To get the best out of these systems, it is important that a symbol table is generated when compiling or linking software, which will provide a cross-reference between labels and symbol names and their physical address in memory. In addition, an assembler source listing which shows the assembler code generated for each line of C or other high level language code is invaluable. Without this information it can be very difficult to use the debugger easily. Having said that, it is quite frustrating having to look up references in very large tables and this highlights one of the restrictions with this type of debugger.

While considered very low level and somewhat limited in their use, onboard debuggers are extremely useful in giving confidence that the board is working correctly and working on an embedded system where an emulator may be impractical. However, this ability to access only at a low level can also place severe limitations on what can be debugged.

The first problem concerns the initialisation routines and in particular the processor's vector table. Breakpoints use either a special breakpoint instruction or an illegal instruction to generate a processor exception when the instruction is executed. Program control is then transferred to the debugger which displays the breakpoint and associated information. Similarly, the debugger may use other vectors to drive the serial port that is connected to the terminal.

This vector table may be overwritten by the initialisation routines of the operating system which can replace them with its own set of vectors. The breakpoint can still be set but when it is reached, the operating system will see it instead of the debugger and not pass control back to it. The system will normally crash because it is not expecting to see a breakpoint or an illegal instruction!

To get around this problem, the operating system may need to be either patched so that its initialisation routine writes the debugger vector into the appropriate location or this must be done using the debugger itself. The operating system is single stepped through its initialisation routine and the instruction that overwrites the vector simply skipped over, thus preserving the debugger's vector. Some operating systems can be configured to preserve the debugger's exception vectors, which removes the need to use the debugger to preserve them.

A second issue is that of memory management where there can be a problem with the address translation. Breakpoints will still work but the addresses returned by the debugger will be physical, while those generated by the symbol table will normally be logical. As a result, it can be very difficult to reconcile the physical address information with the logical information.

The onboard debugger provides a simple but sometimes essential way of debugging VMEbus software. For small amounts of code, it is quite capable of providing a method of debugging which is effective, albeit not as efficient as a full blown symbolic level debugger — or as complex or expensive. It is often the only way of finding out about a system which has hung or crashed.

Task level debugging

In many cases, the use of a low level debugger is not very efficient compared with the type of control that may be needed. A low level debugger is fine for setting a breakpoint at the start of a routine but it cannot set them for particular task functions and operations. It is possible to set a breakpoint at the start of the routine that sends a message, but if only a particular message is required, the low level approach will need manual inspection of all messages to isolate the one that is needed — an often daunting and impractical approach!

To solve this problem, most operating systems provide a task level debugger which works at the operating system level. Breakpoints can be set on system circumstances, such as events, messages, interrupt routines and so on, as well as the more normal memory address. In addition, the ability to filter messages and events is often included. Data on the current executing tasks is provided, such as memory usage, current status and a snapshot of the registers.

Symbolic debug

The ability to use high level language instructions, functions and variables instead of the more normal addresses and their contents is known as symbolic debugging. Instead of using an assembler listing to determine the address of the first instruction of a C function and using this to set a breakpoint,

the symbolic debugger allows the breakpoint to be set by quoting a line reference or the function name. This interaction is far more efficient than working at the assembler level, although it does not necessarily mean losing the ability to go down to this level if needed.

The reason for this is often due to the way that symbolic debuggers work. In simple terms, they are intelligent front ends for assembler level debuggers, where software performs the automatic look-up and conversion between high level language structures and their respective assembler level addresses and contents.

```
12 int prime,count,iter;
13
14 for (iter = 1;iter<=MAX_ITER;iter++)
15 {
16 count = 0;
17 for(i = 0; i<MAX_PRIME; i++)
18 flags[i] = 1;
19 for(i = 0; i<MAX_PRIME; i++)
20 if(flags[i])
21 {
22 prime = i + i + 3;
23 k = i + prime;
24 while (k < MAX_PRIME)
25 {
26 flags[k] = 0;
27 k += prime;
28 }
29 count++;
```

Source code listing with line references

```
000100AA 7C01 MOVEQ #$1,D6
000100AC 7800 MOVEQ #$0,D4
000100AE 7400 MOVEQ #$0,D2
000100B0 207C 0001 2148 MOVEA.L #$12148,A0 000100B6
11BC 0001 2000 MOVE.B #$1,($0,A0,D2.W)
000100BC 5282 ADDQ.L #$1,D2
000100BE 7011 MOVEQ #$11,D0
000100C0 B082 CMP.L D2,D0
000100C2 6EEC BGT.B $100B0
000100C4 7400 MOVEQ #$0,D2 000100C6 207C 0001 2148
MOVEA.L #$12148,A0 000100CC 4A30 2000 TST.B
($0,A0,D2.W) 000100D0 6732 BEQ.B $10104 000100D2
2A02 MOVE.L D2,D5 000100D4 DA82 ADD.L D2,D5 000100D6
5685 ADDQ.L #$3,D5
```

Assembler listing

```
>> 12 int prime,count,iter;
>> 13
- 14 => for (iter = 1;<=iter<=MAX_ITER;iter++)
 000100AA 7C01 MOVEQ #$1,D6
>> 15 {
>> 16 count = 0;
 000100AC 7800 MOVEQ #$0,D4
- 17 => for(i = 0;<= i<MAX_PRIME; i++)  > 000100AE
7400 MOVEQ #$0,D2
>> 18 flags[i] = 1;
 000100B0 207C 0001 2148 MOVEA.L #$12148,A0 {flags}
 000100B6 11BC 0001 2000 MOVE.B #$1,($0,A0,D2.W)
```

```
;- 17 for(i = 0; i<MAX_PRIME; => i++)<=
; 000100BC 5282 ADDQ.L #$1,D2
;- 17 for(i = 0; => i<MAX_PRIME;<=i++)
; 000100BE 7011 MOVEQ #$11,D0
; 000100C0 B082 CMP.L D2,D0
; 000100C2 6EEC BGT.B $100B0
```

Assembler listing with symbolic information

The key to this is the creation of a symbol table which provides the cross-referencing information that is needed. This can either be included within the binary file format used for object and absolute files or, in some cases, stored as a separate file. The important thing to remember is that symbol tables are often not automatically created and, without them, symbolic debug is not possible.

When the file or files are loaded or activated by the debugger, it searches for the symbolic information which is used to display more meaningful information as shown in the various listings. The symbolic information means that breakpoints can be set on language statements as well as individual addresses. Similarly, the code can be traced or stepped through line by line or instruction by instruction.

This has several repercussions. The first is the number of symbolic terms and the storage they require. Large tables can dramatically increase file size and this can pose constraints on linker operation when building an application or a new version of an operating system. If the linker has insufficient space to store the symbol tables while they are being corrected — they are often held in RAM for faster searching and update — the linker may crash with a symbol table overflow error. The solution is to strip out the symbol tables from some of the modules by recompiling them with symbolic debugging disabled or by allocating more storage space to the linker.

The problems may not stop there. If the module is then embedded into a target and symbolic debugging is required, the appropriate symbol tables must be included in the build and this takes up memory space. It is not uncommon for the symbol tables to take up more space than the spare system memory and prevent the system or task from being built or running correctly. The solution is to add more memory or strip out the symbol tables from some of the modules.

It is normal practice to remove all the symbol table information from the final build to save space. If this is done, it will also remove the ability to debug using the symbol information. It is a good idea to have at least a hard copy of the symbol table to help should any debugging be needed.

Emulation

Even using the described techniques, it cannot be stated that there will never be a need for additional help. There will be

```
;- 17 for(i = 0; i<MAX_PRIME; => i++)<=
; 000100BC 5282 ADDQ.L #$1,D2
;- 17 for(i = 0; => i<MAX_PRIME;<=i++)
; 000100BE 7011 MOVEQ #$11,D0
; 000100C0 B082 CMP.L D2,D0
; 000100C2 6EEC BGT.B $100B0
```

Assembler listing with symbolic information

The key to this is the creation of a symbol table which provides the cross-referencing information that is needed. This can either be included within the binary file format used for object and absolute files or, in some cases, stored as a separate file. The important thing to remember is that symbol tables are often not automatically created and, without them, symbolic debug is not possible.

When the file or files are loaded or activated by the debugger, it searches for the symbolic information which is used to display more meaningful information as shown in the various listings. The symbolic information means that breakpoints can be set on language statements as well as individual addresses. Similarly, the code can be traced or stepped through line by line or instruction by instruction.

This has several repercussions. The first is the number of symbolic terms and the storage they require. Large tables can dramatically increase file size and this can pose constraints on linker operation when building an application or a new version of an operating system. If the linker has insufficient space to store the symbol tables while they are being corrected — they are often held in RAM for faster searching and update — the linker may crash with a symbol table overflow error. The solution is to strip out the symbol tables from some of the modules by recompiling them with symbolic debugging disabled or by allocating more storage space to the linker.

The problems may not stop there. If the module is then embedded into a target and symbolic debugging is required, the appropriate symbol tables must be included in the build and this takes up memory space. It is not uncommon for the symbol tables to take up more space than the spare system memory and prevent the system or task from being built or running correctly. The solution is to add more memory or strip out the symbol tables from some of the modules.

It is normal practice to remove all the symbol table information from the final build to save space. If this is done, it will also remove the ability to debug using the symbol information. It is a good idea to have at least a hard copy of the symbol table to help should any debugging be needed.

Emulation

Even using the described techniques, it cannot be stated that there will never be a need for additional help. There will be

times when instrumentation, such as emulation and logic analysis, are necessary to resolve problems within a design quickly. Timing and intermittent problems cannot be easily solved without access to further information about the processor and other system signals. Even so, the recognition of a potential problem source, such as a specific software module or hardware, allows more productive use and a speedier resolution. The adoption of a methodical design approach and the use of ready built boards as the final system, at best remove the need for emulation and, at worst, reduce the amount of time required to debug the system.

There are some problems with using emulation within a board-based system or any rack mounted system. The first is how to get the emulation or logic analysis probe onto the board in the first place. Often the gap between the processor and adjacent boards is too small to cope with the height of the probe. It may be possible to move adjacent boards to other slots, but this can be very difficult or impossible in densely populated racks. The answer is to use an extender board to move the target board out of the rack for easier access. Another problem is the lack of a socketed processor chip which effectively prevents the CPU from being removed and the emulator probe from being plugged in. With the move towards surface mount and high pin count packages, this problem is likely to increase. If you are designing your own board, I would recommend that sockets are used for the processor to allow an emulator to be used. If possible, and the board space allows it, use a zero insertion force socket. Even with low insertion force sockets, the high pin count can make the insertion force quite large. One option that can be used, but only if the hardware has been designed to do so, is to leave the existing processor *in situ* and tri-state all its external signals. The emulator is then connected to the processor bus via another connector or socket and takes over the processor board.

The second problem is the effect that large probes can have on the design especially where high speed buses are used. Large probes and the associated cabling create a lot of additional capacitance loading which can prevent an otherwise sound electronic design from working. As a result, the system speed very often must be downgraded to compensate. This means that the emulator can only work with a slower than originally specified design. If there is a timing problem that only appears while the system is running at high speed, then the emulator is next to useless in providing any help. We will come back to emulation techniques at the end of this chapter.

Optimisation problems

The difficulties do not stop with hardware mechanical problems. Software debugging can be confused or hampered

by optimisation techniques used by the compiler to improve the efficiency of the code. Usually set by options from the command line, the optimisation routines examine the code and change it to improve its efficiency, while retaining its logical design and context. Many different techniques are used but they fall into two main types: those that remove code and those that add code or change it. A compiler may remove variables or routines that are never used or do not return any function. Small loops may be unrolled into straight line code to remove branching delays at the expense of a slightly larger program. Floating point routines may be replaced by inline floating point instructions. The net result is code that is different from the assembler listing produced by the compiler. In addition, the generated symbol table may be radically different from that expected from the source code.

These optimisation techniques can be ruthless; I have known whole routines to be removed and in one case a complete program was reduced to a single NOP instruction! The program was a set of functions that performed benchmark routines but did not use any global information or return any values. The optimiser saw this and decided that as no data was passed to it and it did not modify or return any global data, it effectively did nothing and replaced it with a NOP. When benchmarked, it gave a pretty impressive performance of zero seconds to execute several million calculations.

```
/* sieve.c — Eratosthenes Sieve prime number
calculation */
/* scaled down with MAX_PRIME set to 17 instead of
8091 */

#define MAX_ITER      1
#define MAX_PRIME     17

char      flags[MAX_PRIME];

main ()
{
          register int i,k,l,m;
          int   prime,count,iter;

          for (iter = 1;iter<=MAX_ITER;iter++)
                {
                count = 0;
/* redundant code added here */
                for(l = 0; l < 200; l++ );
                for(m = 128; l > 1; m— );
/* redundant code ends here */
                for(i = 0; i<MAX_PRIME; i++)
                      flags[i] = 1;
                for(i = 0; i<MAX_PRIME; i++)
                      if(flags[i])
                            {
                            prime = i + i + 3;
                            k = i + prime;
                            while (k < MAX_PRIME)
                                  {
```

```
                                        flags[k] = 0;
                                        k += prime;
                                        }
                                    count++;
                                    printf(" prime %d =
%d\n", count, prime);
                                    }
                            }
                        printf("\n%d primes\n",count);
}
```

Source listing for optimisation example

```
                    file    "ctm1AAAa00360"

                    def     aut1.,32
                    def     arg1.,64
                    text
                    global  _main
_main:
                    subu    r31,r31,arg1.
                    st      r1,r31,arg1.-4
                    st      r19,r31,aut1.+0
                    st      r20,r31,aut1.+4
                    st      r21,r31,aut1.+8
                    st      r22,r31,aut1.+12
                    st      r23,r31,aut1.+16
                    st      r24,r31,aut1.+20
                    st      r25,r31,aut1.+24
                    or      r19,r0,1
                    br      @L25
@L26:
                    or      r20,r0,r0
                    or      r23,r0,r0
                    br      @L6
@L7:
                    addu    r23,r23,1
@L6:
                    cmp     r13,r23,200
                    bb1     lt,r13,@L7
                    or      r22,r0,128
                    br      @L10
@L11:
                    subu    r22,r22,1
@L10:
                    cmp     r13,r23,1
                    bb1     gt,r13,@L11
                    or      r25,r0,r0
                    br      @L14
@L15:
                    or.u    r13,r0,hi16(_flags)
                    or      r13,r13,lo16(_flags)
                    or      r12,r0,1
                    st.b    r12,r13,r25
                    addu    r25,r25,1
@L14:
                    cmp     r13,r25,17
                    bb1     lt,r13,@L15
                    or      r25,r0,r0
                    br      @L23
@L24:
                    or.u    r13,r0,hi16(_flags)
                    or      r13,r13,lo16(_flags)
                    ld.b    r13,r13,r25
                    bcnd    eq0,r13,@L17
                    addu    r13,r25,r25
                    addu    r21,r13,3
                    addu    r24,r25,r21
                    br      @L19
@L20:
                    or.u    r13,r0,hi16(_flags)
                    or      r13,r13,lo16(_flags)
                    st.b    r0,r13,r24
                    addu    r24,r24,r21
@L19:
                    cmp     r13,r24,17
                    bb1     lt,r13,@L20
                    addu    r20,r20,1
                    or.u    r2,r0,hi16(@L21)
                    or      r2,r2,lo16(@L21)
                    or      r3,r0,r20
                    or      r4,r0,r21
                    bsr     _printf
@L17:
                    addu    r25,r25,1
@L23:
                    cmp     r13,r25,17
                    bb1     lt,r13,@L24
                    addu    r19,r19,1
@L25:
                    cmp     r13,r19,1
                    bb1     le,r13,@L26
                    or.u    r2,r0,hi16(@L27)
                    or      r2,r2,lo16(@L27)
                    or      r3,r0,r20
                    bsr     _printf
                    ld      r19,r31,aut1.+0
                    ld      r20,r31,aut1.+4
                    ld      r21,r31,aut1.+8
                    ld      r22,r31,aut1.+12
                    ld      r23,r31,aut1.+16
                    ld      r24,r31,aut1.+20
                    ld      r25,r31,aut1.+24
                    ld      r1,r31,arg1.-4
                    addu    r31,r31,arg1.
                    jmp     r1
```

No optimisation

```
                    file    "ctm1AAAa00355"

                    def
                    def     aut1.,32
                    text    arg1.,56
                    global  _main
_main:
                    subu    r31,r31,arg1.
                    st      r1,r31,arg1.-4
                    st.d    r20,r31,aut1.+0
                    st.d    r22,r31,aut1.+8
                    st      r25,r31,aut1.+16
                    or      r20,r0,1
@L26:
                    or      r21,r0,r0
                    or      r25,r0,r0
@L7:
                    addu    r25,r25,1
                    cmp     r13,r25,200
                    bb1     lt,r13,@L7
                    br.n    @L28
                    or      r2,r0,128
@L11:
                    subu    r2,r2,1
@L28:
                    cmp     r13,r25,1
                    bb1     gt,r13,@L11
                    or      r25,r0,r0
                    or.u    r22,r0,hi16(_flags)
                    or      r22,r22,lo16(_flags)
@L15:
                    or      r13,r0,1
                    st.b    r13,r22,r25
                    addu    r25,r25,1
                    cmp     r12,r25,17
                    bb1     lt,r12,@L15
                    or      r25,r0,r0
@L24:
                    ld.b    r12,r22,r25
                    bcnd    eq0,r12,@L17
                    addu    r12,r25,r25
                    addu    r23,r12,3
                    addu    r2,r25,r23
                    cmp     r12,r2,17
                    bb1     ge,r12,@L18
@L20:
                    st.b    r0,r22,r2
                    addu    r2,r2,r23
                    cmp     r13,r2,17
                    bb1     lt,r13,@L20
@L18:
                    addu    r21,r21,1
                    or.u    r2,r0,hi16(@L21)
                    or      r2,r2,lo16(@L21)
                    or      r3,r0,r21
                    bsr.n   _printf
                    or      r4,r0,r23
@L17:
                    addu    r25,r25,1
                    cmp     r13,r25,17
                    bb1     lt,r13,@L24
                    addu    r20,r20,1
                    cmp     r13,r20,1
                    bb1     le,r13,@L26
                    or.u    r2,r0,hi16(@L27)
                    or      r2,r2,lo16(@L27)
                    bsr.n   _printf
                    or      r3,r0,r21
                    ld.d    r20,r31,aut1.+0
                    ld      r1,r31,arg1.-4
                    ld.d    r22,r31,aut1.+8
                    ld      r25,r31,aut1.+16
                    jmp.n   r1
                    addu    r31,r31,arg1.
```

Full optimisation

Assembler listings for optimised and non-optimised compilation

To highlight how optimisation can dramatically change the generated code structure, look at the C source listing for the Eratosthenes Sieve program and the resulting M88000 assembler listings that were generated by using the default non-optimised setting and the full optimisation option. The immediate difference is in the greatly reduced size of the code and the use of the .n suffix with jump and branch instructions to make use of the delay slot. This is a technique used on many RISC processors to prevent a pipeline stall when changing the program flow. If the instruction has a .n suffix, the instruction immediately after it is effectively executed with the branch and not after it, as it might appear from the listing!

In addition, the looping structures have been reorganised to make them more efficient, although the redundant code loops could be encoded simply as a loop with a single branch. If the optimiser is that good, why has it not done this? The reason is that the compiler expects loops to be inserted for a reason and usually some form of work is done within the loop which may change the loop variables. Thus the compiler will take the general case and use that rather than completely remove it or rewrite it. If the loop had been present in a dead code area — within a conditional statement where the conditions would never be met — the compiler would remove the structure completely.

The initialisation routine `_main` is different in that not all the variables are initialised using a store instruction and fetching their values from a stack. The optimised version uses the faster 'or' instruction to set some of the variables to zero.

These and other changes highlight several problems with optimisation. The obvious one is with debugging the code. With the changes to the code, the assembler listing and symbol tables do not match. Where the symbols have been preserved, the code may have dramatically changed. Where the routines have been removed, the symbols and references may not be present. There are several solutions to this. The first is to debug the code with optimisation switched off. This preserves the symbol references but the code will not run at the same speed as the optimised version, and this can lead to some timing problems. A second solution is becoming available from compiler and debugger suppliers, where the optimisation techniques preserve as much of the symbolic information as possible so that function addresses and so on are not lost.

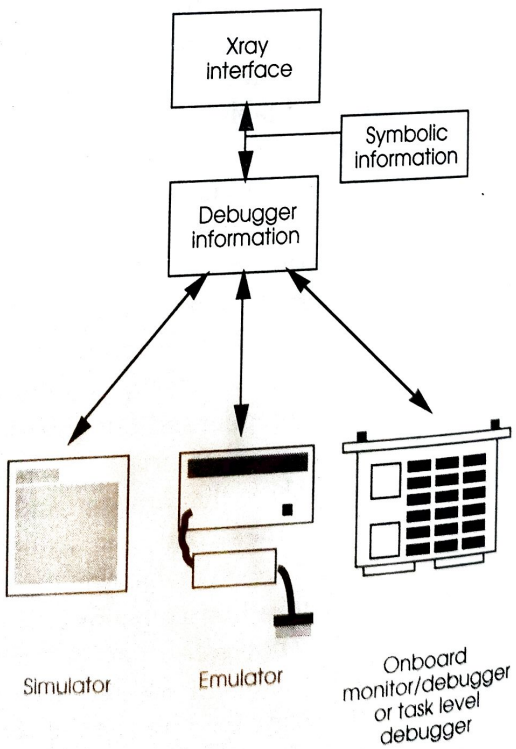
The second issue is concerned with the effect optimisation may have on memory mapped I/O. Unless the optimiser can recognise that a function is dealing with memory mapped I/O, it may not realise that the function is doing some work after all and remove it — with disastrous results. This may require declaring the I/O addresses as a global variable, returning a value at the function's completion or even passing the address to the function itself, so that the optimiser can recognise its true

role. A third complication can arise with optimisations such as unrolling loops and software timing. It is not uncommon to use instruction sequences to delay certain accesses or functions. A peripheral may require a certain number of clock cycles to respond to a command. This delay can be accomplished by executing other instructions, such as a loop or a divide instruction. The optimiser may remove or unroll such loops and replace the inefficient divide instruction with a logical shift. While this does increase the performance, that is not what was required and the delayed peripheral access may not be long enough — again with disastrous results.

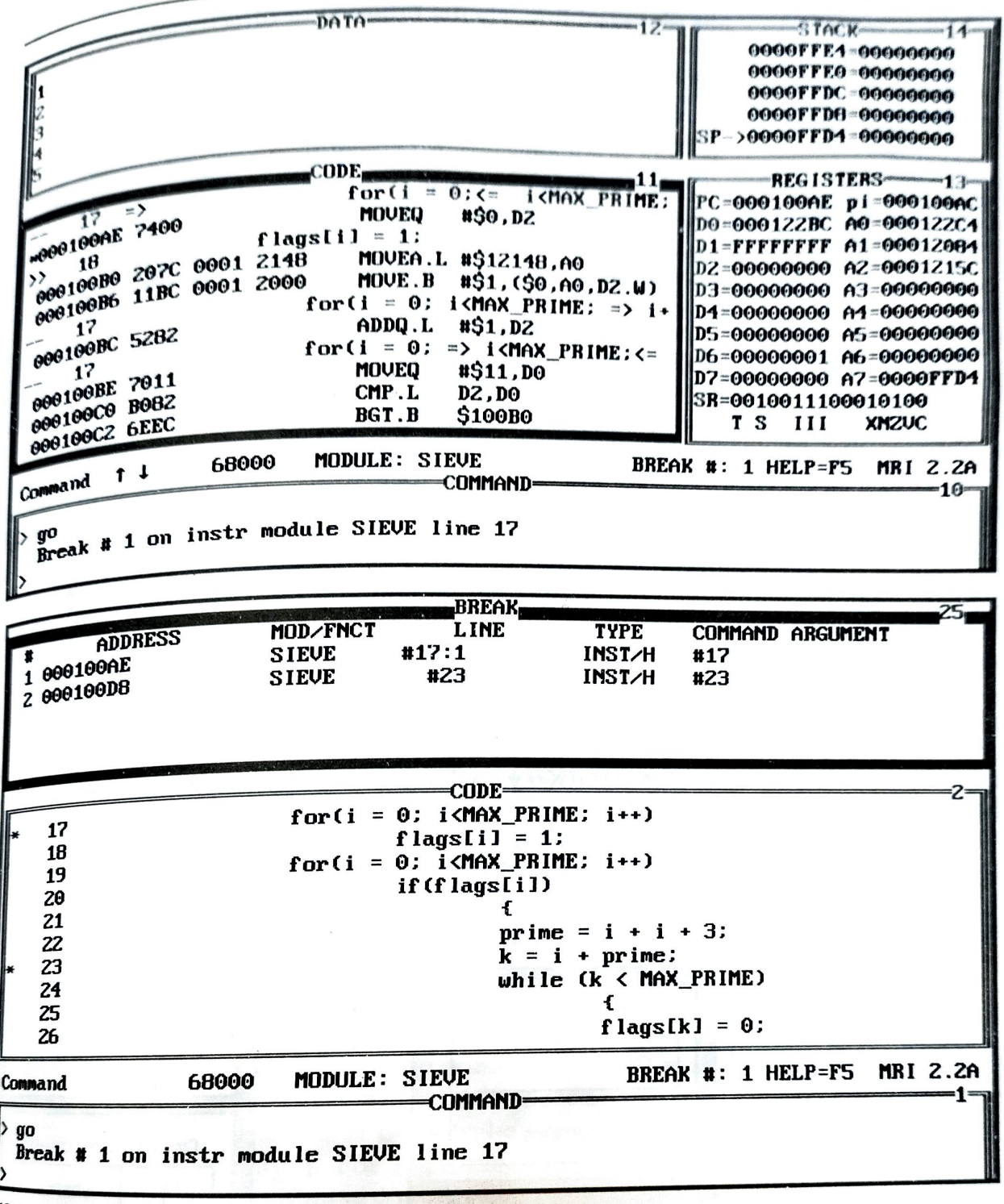
Such software timing should be discouraged not only for this but also for portability reasons. The timing will assume certain characteristics about the processor in terms of processing speed and performance which may not be consistent with other faster board designs or different processor versions.

Xray

It is not uncommon to use all the debugging techniques that have been described so far at various stages of a development. While this itself is not a problem, it has been difficult to get a common set of tools that would allow the various techniques to be used without having to change compilers or libraries, learn different command sets, and so on. The ideal would be a single set of compiler and debugger tools that would work with a simulator, task level debugger, onboard debugger and emulator. This is exactly the idea behind Microtec's Xray product.



Xray structure



Xray screen shots

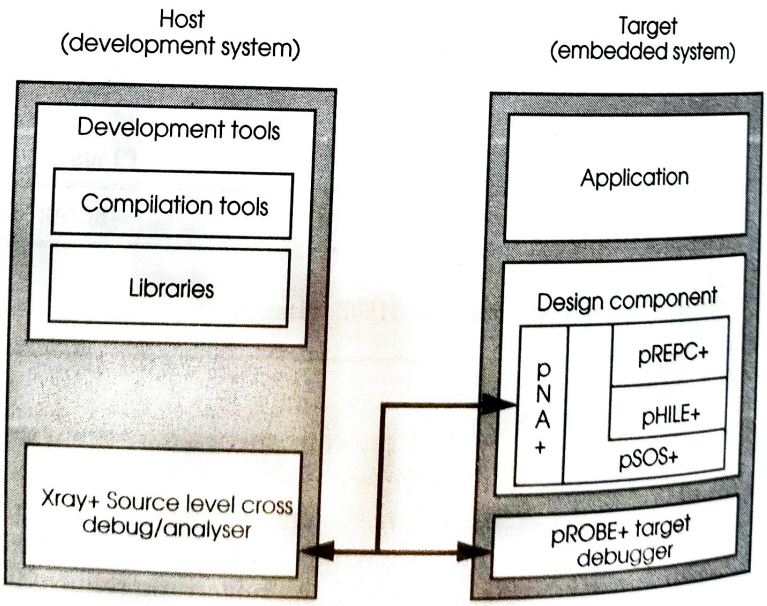
Xray consists of a consistent debugger system that can interface with a simulator, emulator, onboard debugger or operating system task level debugger. It provides a consistent interface which greatly improves the overall productivity because there is no relearning required when moving from one environment to another. It obtains its debugging information from a variety of sources, depending on how the target is being accessed. With the simulator, the information is accessed directly. With an emulator or target hardware, the link is via a

simple serial line, via the Ethernet or directly across a shared memory interface. The data is then used in conjunction with symbolic information to produce the data that the user can see and control on the host machine.

The interface consists of a number of windows which display the debugging information. The windows consist of two types: those that provide core information, such as breakpoints and the processor registers and status. The second type are windows concerned with the different environments, such as task level information. Windows can be chosen and displayed at the touch of a key.

The displays are also consistent over a range of hosts, such as Sun workstations, IBM PCs and UNIX platforms. Either a serial or network link is used to transfer information from the target to the debugger. The one exception is that of the simulator which runs totally on the host system.

So how are these tools used? Xray comes with a set of compiler tools which allows software to be developed on a host system. This system does not have to use the same processor as the target. To execute the code, there is a variety of choices. The simulator is ideal for debugging code at a very early stage, before hardware is available, and allows software development to proceed in parallel with hardware development. Once the hardware is available, the Xray interface can be taken into the target through the use of an emulator or a small onboard debug monitor program. These debug monitors are supplied as part of the Xray package for a particular processor. They can be easily modified to reflect individual memory maps and have drivers from a large range of serial communications peripherals.



pSOS+ debugging

With Xray running in the target, the hardware and initial software routines can be debugged. The power of Xray can be further extended by having an Xray interface from the operat-

ing system debugger. pSOS+ uses this method to provide its debugging interface. This allows task level information to be used to set breakpoints, and so on, while still preserving the lower level facilities. This provides an extremely powerful and flexible way of debugging a target system. Xray has become a *de facto* standard for debugging tools within the real-time and VMEbus market. This type of approach is also being adopted by many other software suppliers.

The role of the development system

An alternative environment for developing software for embedded systems is to use the final operating system as the development environment either on the target system itself or on a similar system. This used to be the traditional way of developing software before the advent of cheap PCs and workstations and integrated cross-compilation.

It still offers distinct advantages over the cross-compilation system. Run-time library support is integrated because the compilers are producing native code. Therefore the run-time libraries that produce executable code on the development system will run unmodified on the target system. The final software can even be tested on the development system before moving it to the target. In addition, the full range of functions and tools can be used to debug the software during this testing phase, which may not be available on the final target. For example, if a target simply used the operating system kernel, it would not have the file system and terminal drivers needed to support an onscreen debugger or help download code quickly from disk to memory. Yet a development system running the full version of the operating system could offer this and other features, such as downloading over a network. However, there are some important points to remember.

Floating point and memory management functions

Floating point co-processors and memory management units should be considered as part of the processor environment and need to be considered when creating code on the target. For example, the development system may have a floating point unit and memory management, while the target does not. Code created on the development system may not run on the target because of these differences. Executing floating point instructions would cause a processor exception while the location of code and data in the memory may not be compatible.

This means that code created for the development system may need recompiling and linking to ensure that the correct run-time routines are used for the target and that they are located correctly. This in turn may mean that the target versions may not run on the development system because its

resources do not match up. This raises the question of the validity of using a development system in the first place. The answer is that the source code and the bulk of the binary code does not need modifying. Calling up a floating point emulation library instead of using floating point instructions will not affect any integer or I/O routines. Linking modules to a different address changes the addresses, not the instructions and so the two versions are still extremely similar. If the code works on the development system, it is likely that it will work on the target system.

While the cross-compilation system is probably the most popular method used today to develop software for embedded systems — due to the widespread availability of PCs and workstations and the improving quality of software tools — it is not the only way. Dedicated development systems can offer faster and easier software creation because of the closer relationship between the development environment and the end target.

Emulation techniques

In-circuit emulation (ICE) has been the traditional method employed to emulate a processor inside an embedded design so that software can be downloaded and debugged *in situ* in the end application. For many processors this is still an appropriate method for debugging embedded systems but the later processors have started to dispense with the emulator as a tool and replace it with alternative approaches.

The main problem is concerned with the physical issues associated with replacing the processor with a probe and cable. These issues have been touched on before but it is worth revisiting them. The problems are:

- Physical limitation of the probe

  With high pin count and high density packages that many processors now use such as quad flat packs, ball grid arrays and so on, the job of getting sockets that can reliably provide good electrical contacts is becoming harder. This is starting to restrict the ability of probe manufacturers to provide headers that will fit these sockets, assuming that the sockets are available in the first place.

  The ability to get several hundred individual signal cables into the probe is also causing problems and this has meant that for some processors, emulators are no longer a practical proposition.

- Matching the electrical characteristics

  This follows on from the previous point. The electrical characteristics of the probe should match that of the device the emulator is emulating. This includes the

electrical characteristics of the pins. The difficulty is that the probe and its associated wiring make this matching very difficult indeed and in some cases, this imposes speed limits on the emulation or forces the insertion of wait states. Either way, the emulation is far from perfect and this can cause restrictions in the use of emulation. In some cases, where speed is of the essence, emulation can prevent the system from working at the correct design speed.

- Field servicing

  This is an important but often neglected point. It is extremely useful for a field engineer to have some form of debug access to a system in the field to help with fault identification and rectification. If this relies on an emulator, this can pose problems of access and even power supplies if the system is remote.

So, faced with these difficulties, many of the more recent processors have adopted different strategies to provide emulation support without having to resort to the traditional emulator and its inherent problems.

The basic methodology is to add some debugging support to the processor that enables a processor to be single stepped and breakpointed under remote control from a workstation or host. This facility is made possible through the provision of dedicated debug ports.

JTAG

JTAG ports were originally designed and standardised to provide a way of taking over the pins of a device to allow different bit patterns to be imposed on the pins allowing other devices within the circuit to be tested. This is important to implement boundary scan techniques without having to remove a processor. It allows access to all the hardware within the system.

The system works by using a serial port and clocking data into a large shift register inside the device. The outputs from the shift register are then used to drive the pins under control from the port.

OnCE

OnCE or on-chip emulation is a debug facility used on Motorola's DSP 56x0x family of DSP chips. It uses a special serial port to access additional registers within the device that provide control over the processor and access to its internal registers. The advantage of this approach is that by bringing out the OnCE port to an external connector, every system can provide its own in circuit emulation facilities by hooking this port to an interface port in a PC or workstation. The OnCE port

allows code to be downloaded and single stepped, breakpoints to be set and the display of the internal registers, even while operating. In some cases, small trace buffers are available to capture key events.

BDM

BDM or background debug mode is provided on Motorola's MC683xx series of processors as well as some of the newer 8 bit microcontrollers such as the MC68HC12. It is similar in concept to OnCE, in that it provides remote control and access over the processor, but the way that it is done is slightly different. The processor has additional circuitry added which provides a special background debug mode where the processor does not execute any code but is under the control of the remote system connected to its BDM port. The BDM state is entered by the assertion of a BDM signal or by executing a special BDM instruction. Once the BDM mode has been entered, low level microcode takes over the processor and allows breakpoints to be set, registers to be accessed and single stepping to take place and so on, under command from the remote host.